Bench Testing Guide

Step-by-step procedures for every hardware test currently outstanding across the HMTL / WLED / esp-now-router ecosystem. Each test states what to connect, the exact commands, what counts as a pass, and — where it matters — what a *false* pass looks like. · Generated 2026-08-29

0. Read this first — four traps that have already cost time

Trap	What to do instead
USB port names reshuffle on every replug. `usbserial-0001` is not a stable identity.	Re-derive port→MAC every session with `esptool chip_id` before flashing. Flashing the wrong board looks like a firmware bug.
NVS shadows compile-time WiFi credentials. Rebuilding with a new SSID changes nothing on its own.	`esptool erase_flash` before flashing any build whose credentials changed. The boot line shows the SSID actually in use.
Re-provisioning is not a liveness check. Sending Improv credentials to an already-connected board knocks it off and reports failure.	Query the device (HTTP/ping) or read the AP client list. Never re-provision to "check" a board.
Absence of traffic is not absence of firmware. Zero received frames is equally "wrong build" and "right build, nothing triggering a send".	Drive a stimulus you control, or read state directly, before concluding anything from silence.

> **Addresses 71 and 72 are the flame-effects trigger boards.** Never send a VALUE or PROGRAM command to them as part of a bench test. Polling is read-only and safe; driving an output is energising flame hardware from a test script. Use any non-flame LED module instead.

1. Range-finding walk — do this before tests 3 and 4

Tests 3 and 4 need the two edge nodes **out of ESP-NOW range of each other** while each still reaches the router. That is a window, not simply "far apart": with the router midway at distance *d* from each edge, you need **R/2 < d < R**, where R is the real range of these boards. Estimates for R differ by more than 10× (a few hundred metres from typical ESP-NOW figures; roughly 20 m from this project's own bench RSSI reading), so measure it rather than assume.

1. Flash **two** boards with the plain router build:

```
cd WLED_dev/esp-now-router
PLATFORMIO_BUILD_FLAGS='-DROUTER_WIFI_SSID=\"Lightbringer\" -DROUTER_WIFI_PASS=\"Morningstar\"' \
  pio run -e esp32dev --target upload --upload-port /dev/cu.usbserial-XXXX
```

2. Leave board **A** at the bench on USB. Put board **B** on a battery.
3. Walk B away in a straight line, stopping every 25 m (or every 5 m if you expect the short estimate).
4. At each stop read board A — **A does the measuring**, because RSSI is recorded by the receiver. B only needs power, so no laptop travels.

```
curl -s http://<A-ip>/routes   # rssiDirect per origin — device-to-device, not the AP path
curl -s http://<A-ip>/debug    # rxFrames should keep climbing while B is heard
```

5. The distance at which `rxFrames` stops advancing for B is **R**.

> **Result to record:** R in metres, plus the RSSI curve on the way out. Then place each edge at roughly **0.7 R** for tests 3 and 4 — comfortably inside the window at both ends.

> If Lightbringer is down there is no HTTP. In that case the serial heartbeat on A shows WiFi state and channel but **not** receive counts, so it cannot tell you whether B is still heard. Ask for the 3-line `ROUTER_CHANNEL_DIAG` addition that prints `rxFrames` and the most recent origin RSSI before walking without a network.

2. ESP-NOW starts on the AP channel — closes the last open row on esp-now-router #6

Highest value per minute: ~10 minutes, one board. This is the only claim on already-merged code that has not been re-verified since the fix was reworked.

Needs: one ESP32 devkit on USB · Lightbringer powered on.

1. Identify the board (never trust the port name):

```
ET=~/.platformio/packages/tool-esptoolpy/esptool.py
PY=~/.platformio/penv/bin/python
$PY $ET --port /dev/cu.usbserial-XXXX chip_id      # note the MAC
```

2. Erase, then flash the merged router build with credentials:

```
$PY $ET --port /dev/cu.usbserial-XXXX erase_flash
cd WLED_dev/esp-now-router
PLATFORMIO_BUILD_FLAGS='-DROUTER_WIFI_SSID=\"Lightbringer\" -DROUTER_WIFI_PASS=\"Morningstar\" -
DROUTER_CHANNEL_DIAG' \
  pio run -e esp32dev --target upload --upload-port /dev/cu.usbserial-XXXX
```

3. Watch the console from reset for 90 seconds.

> **PASS:** one boot banner; a line `espnow: started, associated=1 channel=N` where N is Lightbringer's channel; **zero** `attach: advert send failed` in 90 s; the diag line shows `WiFi.status=3` with a real IP.

> **FAIL modes worth telling apart:** `rc=-3` is queue-full, meaning the radio is not transmitting — the original bug. `WiFi.status=1` means the SSID was not found (the AP is off or out of range), which is **not** a firmware failure. `status=6` means found but not associated — check the password.

3. Multi-hop relay — mesh backbone item 2

Needs: three boards (1 router + 2 WLED edges on the `ampworks` build) · the separation from test 1 · Lightbringer up.

1. Flash the router as in test 2. Flash both edges with the WLED `ampworks` env. **Note:** `[env:ampworks]` cannot be USB-flashed by `pio -t upload` — it OTAs to a hardcoded IP. Flash the artifacts directly:

```
$PY $ET --chip esp32 --port /dev/cu.usbserial-XXXX --baud 460800 write_flash \
  0x1000 bootloader.bin 0x8000 partitions.bin 0xe000 boot_app0.bin 0x10000 firmware.bin
```

2. Provision both edges to Lightbringer over Improv Serial (needs the AP up).
3. Place edge A and edge C at ~0.7 R either side of the router; confirm with test 1's method that **A cannot hear C directly**. This is the precondition — without it the test passes vacuously.
4. **Negative control first:** power the router **down**. Drive a state change on A. Confirm C does **not** follow.
5. Power the router up. Repeat the same drive on A.

> **PASS:** with the router down C does not change; with it up C follows. Delivery is loop-free and there is no duplicate wave.

> Skipping step 4 is the classic way to record a false pass — if A and C can hear each other directly, C follows whether or not the backbone works.

4. Multi-hop control — mesh backbone item 8

Needs: same rig as test 3, plus a phone on the network.

1. With the topology of test 3 verified, change a preset or effect from a phone attached to node A.
2. Confirm every other node follows, including C across the router hop.
3. **Roaming:** move the phone to a different node mid-session; confirm control still works and the installation does not revert to older state.

> **PASS:** fan-out reaches C via the backbone; roaming does not regress state.

5. Sensor-path regression — mesh backbone item 9

Needs: an **MPR121** attached to an edge. Without one this test cannot fail and therefore proves nothing: `broadcastLocalState()` returns early with no sensor present.

1. Attach the MPR121 to one edge; confirm touch events register locally first.
2. Run control traffic (test 4) while generating touch events.
3. Watch for sensor events being dropped or delayed — they share the 64-byte slot ring and channel with control traffic.

> **PASS:** touch-sync still behaves while control traffic flows.

6. RS485 load and hitch watch — the one step still open on the bridge

Steps 5–9 of RS485 bring-up are already discharged by the 2026-08-19 bench session (UDP → bridge(96) → RS485 @28000 → unreflashed module 72 → poll response relayed back, VALUE handled, counters agreeing). Only the load test remains, and it is the most valuable one left.

Needs: the led_driver board with its RS485 transceiver populated · a **non-flame** HMTL module on the bus to answer · both on the bus at 28000 baud.

1. Record counters and the RS485 framing-error count **before** starting.
2. Hold sustained UDP command traffic at the bridge — target 200 sustained sends — with the LEDs running a busy effect.
3. Watch throughout for: reboots, watchdog resets, visible LED-refresh hitching, and tx-drop counts.
4. Record counters and framing errors **after**.

> **PASS:** no reboot, no watchdog, bounded tx-drop, no visible stutter, and framing errors **unchanged** — not merely "nothing looked wrong".

> **Why this one matters:** the same bench log recorded "one frame of a 4-frame burst (150 ms spacing) silently lost, succeeded on retry" — roughly 25% loss at a gentle rate. `sendMsgTo` blocks ~48 ms per frame at 28000 baud and is rate-limited to one frame per `loop()`. This test is the only check that says whether that is the rate limiter behaving as designed or something worse under load.

7. HMTL-over-HTTP bridge — the `POST /hmtl` path

Needs: the same rig as test 6. Not covered by the 2026-08-19 run, which was UDP.

1. **Control first:** confirm the module answers a plain **UDP** poll. If UDP cannot reach it, an HTTP failure says nothing about the bridge.
2. POST a value command to a **non-flame** LED module via `POST /hmtl`; confirm the LED **physically changes**.
3. POST a POLL; confirm the response comes back in the HTTP reply and matches **byte-for-byte** what the UDP path returns for the same poll.
4. Issue an HTTP command while a UDP burst is in flight; record framing-error counts before and after.

> **An HTTP 200 is not evidence.** It is a value the endpoint controls and would pass with the bus unplugged. The physical LED change and the byte-identical reply are the claims.

8. LoRa RFM95 bench — esp-now-router Task A

Needs: one SparkFun ESP32 LoRa 1-CH Gateway (WRL-15006) for item 1; a **second** for item 2. Three exist in the ecosystem (addresses 96, 97, 129) — 129 is the fire controller.

> **Build the right target.** Use `ArduinoLibs/platformio/RFM95Socket/RFM95_SocketTest`, env `esp32_lora_gw_a`. **Not** HMTL_Module's `esp32_lora_gw` — nothing in HMTL_Module includes `LoRa.h`, so that env compiles none of the library and comes up with no radio, presenting exactly like a hardware fault.

1. **Item 1 (one board):** flash `esp32_lora_gw_a`; confirm `initialized()` returns true, the pin map is right, and there is no boot loop.
2. **Item 2 (two boards):** flash the peer with `esp32_lora_gw_b` — same sketch, addresses swapped. Send from A to B and confirm B **accepts** a frame addressed to it and **ignores** one addressed elsewhere. The negative half is the whole point: LoRa has no addressing of its own, and the software filter is all that stops every node acting on every packet.
3. **Item 3:** confirm RS485 still works with the radio active on the same board.

> **Do not over-read a green.** `RFM95_SocketTest` is standalone and WLED-free, so the global-SPI collision does not apply to it. Passing proves the pin map and the radio; it does **not** license assuming the WLED path works, where the WS2801 bus claims VSPI first.

Sources: esp-now-router #6/#7 · docs/led-driver-esp32/BENCH-LOG.md (2026-08-19) · docs/fire-controller-esp32/BENCH-LOG.md · WLED_dev todo plans · mesh exchange with the mac-mini todo-handler agent, 2026-08-28/29.